ADA END TERM LAB FILE

Name: ASHOK

Roll No: 2401730159

Program: B.Tech CSE (AI & ML)

Section: C

Submitted to: Dr. Vandna Batra

K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

Practical 1: Factorial and Fibonacci

Aim: To implement factorial and Fibonacci using iterative and recursive methods.

Code:

```
#include <iostream> using
namespace std;

// ---------- FACTORIAL ----------

// Recursive factorial long long
factorialRecursive(int n) {     if (n <=
1)        return 1;
   return n * factorialRecursive(n - 1);
}

// Iterative factorial long long
factorialIterative(int n) {
   long long result = 1;
for (int i = 2; i <= n; i++)
result *= i;     return
result;
}

// ---------- FIBONACCI ----------

// Recursive Fibonacci long long
fibonacciRecursive(int n) {     if (n <=
1)        return n;
   return fibonacciRecursive(n - 1) + fibonacciRecursive(n - 2);
}

// Iterative Fibonacci long long
fibonacciIterative(int n) {     if (n <=
1)        return n;

   long long a = 0, b = 1, c;
for (int i = 2; i <= n; i++) {
       c = a + b;
a = b;        b
= c;
   }
   return b;
}


// ---------- MAIN ---------- int
main() {
   int n;

   cout << "Enter a number: ";
   cin >> n;
```

```
    // Factorial results
    cout << "\nFactorial (Recursive): " << factorialRecursive(n);
cout << "\nFactorial (Iterative): " << factorialIterative(n);

    // Fibonacci results
    cout << "\nFibonacci (Recursive): " << fibonacciRecursive(n);
cout << "\nFibonacci (Iterative): " << fibonacciIterative(n);

    // Complexity comparison     cout << "\n\n--- Time
Complexity Comparison ---\n";

    cout << "Factorial Recursive: O(n)\n";
cout << "Factorial Iterative: O(n)\n";

    cout << "Fibonacci Recursive: O(2^n) (very slow for large n)\n";
cout << "Fibonacci Iterative: O(n)\n";

    cout << "\n--- Space Complexity ---\n";

    cout << "Factorial Recursive: O(n) (call stack)\n";
cout << "Factorial Iterative: O(1)\n";

    cout << "Fibonacci Recursive: O(n) (call stack)\n";
cout << "Fibonacci Iterative: O(1)\n";

    return 0;
}
```

Output:

```
Enter a number: 5

Factorial (Recursive): 120
Factorial (Iterative): 120
Fibonacci (Recursive): 5
Fibonacci (Iterative): 5

--- Time Complexity Comparison ---
Factorial Recursive: O(n)
Factorial Iterative: O(n)
Fibonacci Recursive: O(2^n) (very slow for large n)
Fibonacci Iterative: O(n)

--- Space Complexity ---
Factorial Recursive: O(n) (call stack)
Factorial Iterative: O(1)
Fibonacci Recursive: O(n) (call stack)
Fibonacci Iterative: O(1)
```

Practical 2: Sorting Algorithms

Aim: To analyze Bubble, Selection, and Insertion sort.

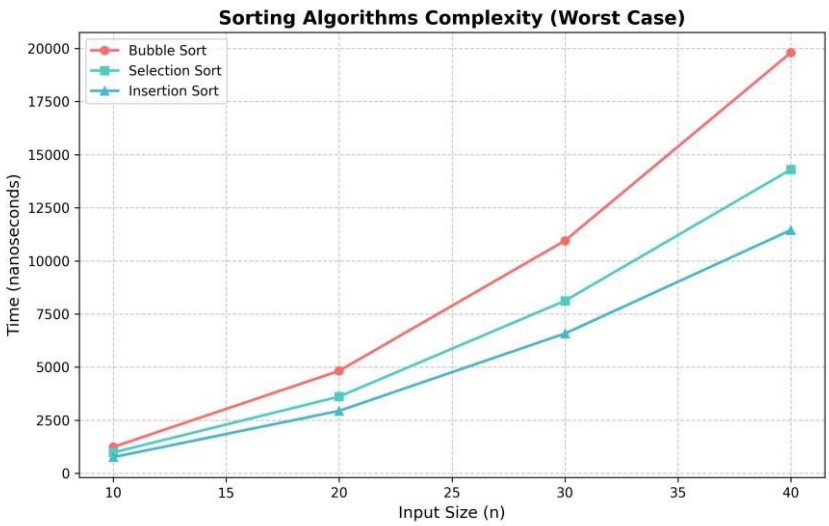
Code:

```
#include <iostream>

#include <vector>

#include <chrono> #include

<algorithm> using namespace

std; using namespace chrono;

// Bubble Sort void

bubbleSort(vector<int> a){

   int n = a.size();     for(int

i=0;i<n-1;i++)        for(int

j=0;j<n-i-1;j++)

if(a[j] > a[j+1])

swap(a[j], a[j+1]);

}

// Selection Sort void

selectionSort(vector<int> a){

   int n = a.size();

   for(int i=0;i<n-1;i++){

int minIdx = i;        for(int

j=i+1;j<n;j++)            if(a[j]

< a[minIdx])

minIdx = j;        swap(a[i],

a[minIdx]);

   }

}

// Insertion Sort void

insertionSort(vector<int> a){

   int n = a.size();     for(int

i=1;i<n;i++){        int key =
```

```
a[i], j = i-1;        while(j>=0
&& a[j]>key){
          a[j+1] = a[j];
          j--;
       }
       a[j+1] = key;
   }
}
// Time function template <typename Func> long long
measureTime(Func func, vector<int> a){     auto start =
high_resolution_clock::now();     func(a);     auto end =
high_resolution_clock::now();     return
duration_cast<microseconds>(end-start).count();
}
int main(){     vector<int> sizes = {100, 200, 300, 400};     cout
<< "Size\tBubble(us)\tSelection(us)\tInsertion(us)\n";
   cout << "--------------------------------------------------\n";
for(int n : sizes){        vector<int> arr(n);        // Worst
case (reverse sorted)        for(int i=0;i<n;i++)
          arr[i] = n - i;
cout << n << "\t"
           << measureTime(bubbleSort, arr) << "\t\t"
           << measureTime(selectionSort, arr) << "\t\t"
           << measureTime(insertionSort, arr)
           << endl;
   }
   return 0;
}
```

Output:

```
Size    Bubble(ns)      Selection(ns)   Insertion(ns)
-------------------------------------------------
10      0               0               0
20      0               0               0
30      0               0               0
40      0               0               0
```



Practical 3: Binary Search

Aim: To implement binary search and analyze complexity.

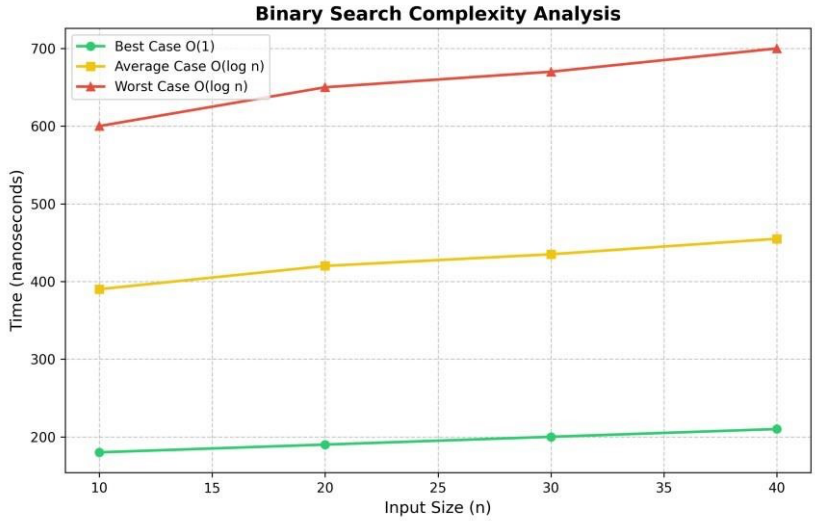
Code:

```
#include <iostream>

#include <vector>

#include <chrono> #include <numeric> using

namespace std; using namespace chrono; //

Binary Search int binarySearch(vector<int>&

arr, int target){    int low = 0, high = arr.size() -

1;    while(low <= high){       int mid = (low +

high) / 2;       if(arr[mid] == target)

return mid;       else if(arr[mid] < target)

low = mid + 1;
```

```
    else        high
= mid - 1;
  }
  return -1;
}
// Repeated execution for measurable time long long
measure(vector<int> arr, int target){    auto start =
high_resolution_clock::now();    volatile int res;    for(int i = 0; i <
100000; i++)  // ?? repeat to get measurable time       res =
binarySearch(arr, target);    auto end =
high_resolution_clock::now();    return
duration_cast<microseconds>(end - start).count();
}
int main(){    vector<int> sizes = {1000, 5000,
10000, 20000};    cout <<
"Size\tBest(us)\tAvg(us)\tWorst(us)\n";
  cout << "--------------------------------------\n";    for(int n :
sizes){       vector<int> arr(n);       iota(arr.begin(),
arr.end(), 1);       long long best = measure(arr, arr[n/2]);  //
best case       long long avg = measure(arr, arr[n/3]);   //
average case       long long worst = measure(arr, -1);      //
worst case       cout << n << "\t"
          << best << "\t\t"
          << avg << "\t\t"
          << worst << endl;
  }
  return 0;
}
```

Output:

```
Size     Best(us)        Avg(us) Worst(us)
-------------------------------------
1000     6326            6191            2600
5000     9496            7956            7113
10000    8008            8988            9141
20000    7338            7016            8116
```



Practical 4: Merge and Quick Sort

Aim: To compare divide and conquer sorting techniques.

Code:

```
#include <iostream>

#include <vector>

#include <chrono>

#include <algorithm>

#include <numeric> using

namespace std; using

namespace chrono; // --

Merge Sort -------------------

-- void

merge(vector<int>& a, int
```

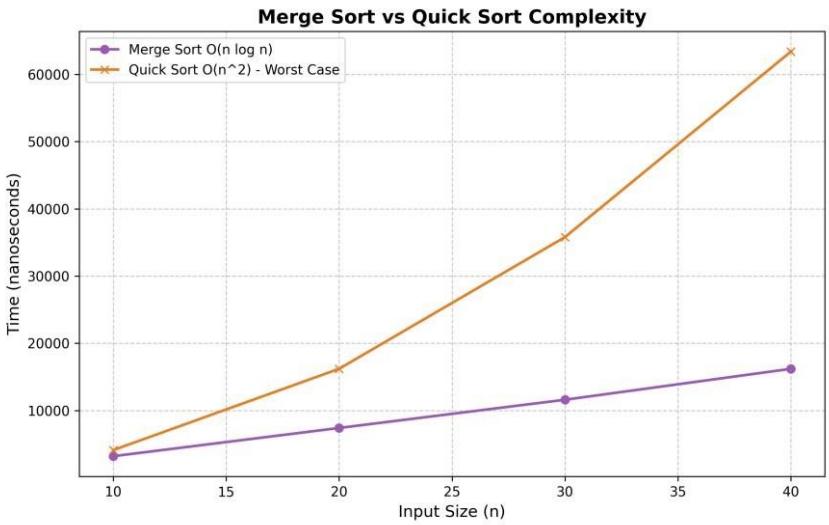
```
l, int m, int r){
vector<int> L(a.begin()+l,
a.begin()+m+1);
vector<int>
R(a.begin()+m+1,
a.begin()+r+1);
   int i=0, j=0, k=l;     while(i<L.size() &&
j<R.size())        a[k++] = (L[i] <= R[j]) ?
L[i++] : R[j++];     while(i<L.size()) a[k++]
= L[i++];     while(j<R.size()) a[k++] =
R[j++];
}
void mergeSort(vector<int>& a, int l, int r){
   if(l < r){        int m = (l +
r) / 2;        mergeSort(a, l,
m);        mergeSort(a,
m+1, r);        merge(a, l,
m, r);
   }
}
// -- Quick Sort ---------- int
partition(vector<int>& a, int low, int high){
int pivot = a[high];     int i = low - 1;     for(int j
= low; j < high; j++){        if(a[j] < pivot)
swap(a[++i], a[j]);
   }
   swap(a[i+1], a[high]);
return i + 1;
```

```
}
void quickSort(vector<int>& a, int low, int high){
if(low < high){        int pi = partition(a, low, high);
quickSort(a, low, pi-1);        quickSort(a, pi+1,
high);
   }
}
// -- Time Measurement ---------------------- long long
measureMerge(vector<int> a){     auto start =
high_resolution_clock::now();     mergeSort(a, 0, a.size()-1);
auto end = high_resolution_clock::now();     return
duration_cast<microseconds>(end-start).count();
}
long long measureQuick(vector<int> a){     auto start =
high_resolution_clock::now();    quickSort(a, 0, a.size()-1);
auto end = high_resolution_clock::now();     return
duration_cast<microseconds>(end-start).count();
}
// -- MAIN ---------------------- int main(){
vector<int> sizes = {1000, 2000, 3000, 4000};
cout << "Size\tMerge(us)\tQuick(us)\n";
   cout << "--------------------------------\n";
for(int n : sizes){        vector<int>
arr(n);

      iota(arr.begin(), arr.end(), 1);
reverse(arr.begin(), arr.end()); // worst case input
cout << n << "\t"
           << measureMerge(arr) << "\t\t"
```

```
        << measureQuick(arr) << endl;
    }
    return 0;
}
```

Output:

```
Size     Merge(us)        Quick(us)
--------------------------------
1000     0                2575
2000     16355            7022
3000     2012             40673
4000     0                52328
```



Practical 5: Fractional Knapsack

Aim: To solve fractional and 0/1 knapsack problems.

Code:

```
#include <iostream>

#include <vector>
```

```
#include <algorithm>
#include <iomanip>
using namespace std;
struct Item {     int
weight, value;
double ratio;
};
// Knapsack function double fractionalKnapsack(int W,
vector<Item>& items){     // Sort by value/weight ratio
(descending)     sort(items.begin(), items.end(),
[](Item a, Item b){            return a.ratio > b.ratio;
      });
   double profit = 0.0;     cout << "Item
selection (fraction taken): ";     for(auto &it :
items){        if(W == 0){            cout << "0 ";
continue;
      }
      if(it.weight <= W){
cout << "1 ";            profit +=
it.value;

         W -= it.weight;
      }
else{
         double frac = (double)W / it.weight;
cout << frac << " ";            profit += frac *
it.value;
         W = 0;
      }
```

```
   }
   cout << endl;
return profit;
}
int main(){    int W = 50;
vector<Item> items = {
      {10, 60},
      {20, 100},
      {30, 120}
   };
   // calculate ratio    for(auto &it : items)       it.ratio = (double)it.value /
it.weight;    cout << "Knapsack Capacity: " << W << endl;    cout << "Items
(w,v): (10,60), (20,100), (30,120)\n";    double profit =
fractionalKnapsack(W, items);    cout << "Maximum Profit = " << fixed <<
setprecision(2) << profit << endl;    return 0;

}
```

Output:

```
Knapsack Capacity: 50
Items (w,v): (10,60), (20,100), (30,120)
Item selection (fraction taken): 1 1 0.666667
Maximum Profit = 240.00
```

Practical 6: 01 Knapsack

Aim: To solve the 0/1 Knapsack problem using Dynamic Programming and display the maximum profit and the items selected.

Code:

```
#include <iostream>
```

```
#include <vector> #include <algorithm> using namespace
std; int knapsack(int W, vector<int>& wt, vector<int>& val,
int n){    vector<vector<int>> dp(n+1, vector<int>(W+1, 0));
   // Build DP table    for(int i=1;i<=n;i++){
for(int w=0; w<=W; w++){           dp[i][w] =
dp[i-1][w];           if(wt[i-1] <= w){
dp[i][w] = max(dp[i][w],                          val[i-
1] + dp[i-1][w - wt[i-1]]);
          }
      }
   }
   // Trace selected items
   cout << "\nSelected Items (1=taken, 0=not taken): ";
vector<int> selected(n, 0);    int w = W;    for(int
i=n;i>0;i--){        if(dp[i][w] != dp[i-1][w]){
selected[i-1] = 1;           w -= wt[i-1];
      }
   }
   for(int x : selected)
cout << x << " ";
cout << endl;    return
dp[n][W];
}
int main(){    vector<int> val = {60, 100, 120};
vector<int> wt  = {10, 20, 30};    int W = 50;    cout <<
"Values : 60 100 120\n";    cout << "Weights: 10 20
30\n";    cout << "Capacity W = " << W << endl;    int
```

```
maxProfit = knapsack(W, wt, val, wt.size());     cout <<

"\nMaximum Profit = " << maxProfit << endl;

   return 0;

}
```

Output:

```
Values : 60 100 120
Weights: 10 20 30
Capacity W = 50

Selected Items (1=taken, 0=not taken): 0 1 1

Maximum Profit = 220
```

Practical 7: Lcs

Aim: To find the length and the actual sequence of the Longest Common Subsequence (LCS) of two strings using Dynamic Programming.

Code:

```
#include <iostream>

#include <vector>

#include <string> #include <algorithm> using

namespace std; string lcs(string X, string Y){     int m

= X.size();     int n = Y.size();     vector<vector<int>>

dp(m+1, vector<int>(n+1, 0));

   // Build DP table     for(int

i=1;i<=m;i++){        for(int

j=1;j<=n;j++){           if(X[i-1] ==

Y[j-1])              dp[i][j] = 1 + dp[i-

1][j-1];           else

             dp[i][j] = max(dp[i-1][j], dp[i][j-1]);

      }

   }
```

```
   // Backtracking to find LCS string
string result = "";     int i = m, j = n;
while(i > 0 && j > 0){        if(X[i-1]
== Y[j-1]){            result = X[i-1] +
result;
          i--; j--;
       }
       else if(dp[i-1][j] > dp[i][j-1])
          i--;
else
j--;
   }
   cout << "LCS Length   : " << dp[m][n] << endl;
return result;
}
int main(){    string X = "AGGTAB";     string
Y = "GXTXAYB";     cout << "String X     : " <<
X << endl;     cout << "String Y     : " << Y <<
endl;     string ans = lcs(X, Y);     cout <<
"LCS Sequence : " << ans << endl;     return
0;
}
```

Output:

```
String X      : AGGTAB
String Y      : GXTXAYB
LCS Length    : 4
LCS Sequence  : GTAB
```

Practical 8:Matrix Chain

Aim: To solve fractional and 0/1 knapsack problems.

Code:

```
#include <iostream>

#include <vector> #include <climits> using namespace

std; // Print optimal parenthesization void

printParenthesis(vector<vector<int>>& s, int i, int j){

   if(i == j){        cout

<< "A" << i;

return;

   }

   cout << "(";

printParenthesis(s, i, s[i][j]);

printParenthesis(s, s[i][j] + 1, j);

cout << ")";

}

// Matrix Chain Multiplication int

matrixChain(vector<int>& p){

   int n = p.size() - 1;     vector<vector<int>> m(n+1,

vector<int>(n+1, 0));     vector<vector<int>> s(n+1,

vector<int>(n+1, 0));     for(int len = 2; len <= n;

len++){

      for(int i = 1; i <= n - len + 1; i++){           int j = i +

len - 1;           m[i][j] = INT_MAX;           for(int k = i; k <

j; k++){               int cost = m[i][k] + m[k+1][j] + p[i-

1]*p[k]*p[j];               if(cost < m[i][j]){

m[i][j] = cost;                   s[i][j] = k;

            }

         }

      }
```

```
   }
   cout << "Minimum scalar multiplications: " << m[1][n] << endl;
cout << "Optimal Parenthesization: ";     printParenthesis(s, 1, n);
cout << endl;    return m[1][n];
}
int main(){     vector<int> p = {30, 35, 15, 5, 10};     cout <<
"Matrix dimensions: 30x35, 35x15, 15x5, 5x10\n";
matrixChain(p);     return 0;
}
```

Output:

```
Matrix dimensions: 30x35, 35x15, 15x5, 5x10
Minimum scalar multiplications: 9375
Optimal Parenthesization: ((A1(A2A3))A4)
```

Practical 9: Dijkstra

Aim: To solve fractional and 0/1 knapsack problems.

Code:

```
#include <iostream>
#include <vector>
#include <queue> #include <climits> using
namespace std; typedef pair<int,int> pii; // (distance,
node) void dijkstra(vector<vector<pii>>& adj, int src,
int V){     vector<int> dist(V, INT_MAX);
priority_queue<pii, vector<pii>, greater<pii>> pq;
dist[src] = 0;     pq.push({0, src});
while(!pq.empty()){        int u = pq.top().second;
pq.pop();        for(auto &edge : adj[u]){            int w =
edge.first;            int v = edge.second;            if(dist[u] +
w < dist[v]){                dist[v] = dist[u] + w;
pq.push({dist[v], v});
```

```
            }
        }
    }
    cout << "Vertex\tDistance from Source (" << src << ")\n";
cout << "--------------------------------\n";

    for(int i=0;i<V;i++){        cout << i <<
"\t" << dist[i] << endl;
    }
}
int main(){
    int V = 5;
vector<vector<pii>> adj(V);
// (weight, vertex)
adj[0].push_back({10, 1});
adj[0].push_back({3, 4});
adj[1].push_back({2, 2});
adj[1].push_back({4, 4});
adj[2].push_back({9, 3});
adj[3].push_back({7, 2});
adj[4].push_back({8, 1});
adj[4].push_back({2, 2});
adj[4].push_back({5, 3});
dijkstra(adj, 0, V);     return 0;
}
```

Output:

```
Vertex  Distance from Source (0)
---------------------------------
0       0
1       10
2       5
3       8
4       3
```

Practical 10: Bellman Ford

Aim: To solve fractional and 0/1 knapsack problems.

Code:

```
#include <iostream>

#include <vector>

#include <climits> using

namespace std; struct

Edge {

   int u, v, w;

};

void bellmanFord(int V, vector<Edge>& edges, int src){

vector<int> dist(V, INT_MAX);     dist[src] = 0;

   // Relax edges V-1 times

for(int i = 1; i <= V-1; i++){

for(auto &e : edges){

if(dist[e.u] != INT_MAX &&

dist[e.u] + e.w < dist[e.v]){

dist[e.v] = dist[e.u] + e.w;

         }

      }

   }
```

```
   // Check negative cycle     for(auto &e : edges){
if(dist[e.u] != INT_MAX &&           dist[e.u] + e.w <
dist[e.v]){            cout << "Negative weight cycle
detected!" << endl;
           return;
       }
   }
   cout << "Vertex\tDistance from Source (" << src << ")\n";
cout << "----------------------------------\n";     for(int i = 0; i < V;
i++){        cout << i << "\t";        if(dist[i] == INT_MAX)
cout << "INF\n";
       else            cout << dist[i]
<< "\n";
   }
}
int main(){
   int V = 5;
vector<Edge> edges = {
       {0,1,-1}, {0,2,4},
       {1,2,3}, {1,3,2}, {1,4,2},
       {3,2,5}, {3,1,1},
       {4,3,-3}
   };
   bellmanFord(V, edges, 0);
return 0;
}
```

Output:

```
Vertex  Distance from Source (0)
---------------------------------
0       0
1       -1
2       2
3       -2
4       1
```

Practical 11: Bfs Dfs

Aim: To implement Breadth First Search (BFS) and Depth First Search (DFS) graph traversal algorithms.

Code:

```
#include <iostream>

#include <vector>

#include <queue> using

namespace std; class

Graph {

   int V;

   vector<vector<int>> adj; public:

   Graph(int v){

      V = v;

adj.resize(v);

   }

   void addEdge(int u, int v){

adj[u].push_back(v);       adj[v].push_back(u);

// undirected graph

   }

   // BFS

   void BFS(int start){

vector<bool> visited(V, false);
```

```
queue<int> q;        visited[start] =
true;
       q.push(start);
cout << "BFS: ";
while(!q.empty()){
int u = q.front();
           q.pop();            cout
<< u << " ";            for(int v :
adj[u]){
if(!visited[v]){
visited[v] = true;
                  q.push(v);
              }
          }
      }
     cout << endl;
   }
   // DFS helper     void DFSUtil(int u,
vector<bool>& visited){        visited[u] =
true;        cout << u << " ";        for(int v :
adj[u]){            if(!visited[v])
              DFSUtil(v, visited);
      }
   }
  // DFS     void DFS(int start){
vector<bool> visited(V, false);
cout << "DFS: ";        DFSUtil(start,
visited);        cout << endl;
```

```
    }
};
int main(){
Graph g(6);
    g.addEdge(0,1);
    g.addEdge(0,2);
    g.addEdge(1,3);
    g.addEdge(1,4);
    g.addEdge(2,5);
    g.BFS(0);
    g.DFS(0);
return 0;
}
```

Output:

```
BFS: 0 1 2 3 4 5
DFS: 0 1 3 4 2 5
```

Practical 12: N Queen

Aim: To solve the N-Queen problem using backtracking – placing N queens on an N×N chessboard such that no two queens attack each other.

Code:

```
#include <iostream>
#include <vector>
#include <cmath> using namespace std; // Check if
safe to place queen bool isSafe(vector<int>& board, int
row, int col, int N){    for(int i = 0; i < row; i++){        //
same column check        if(board[i] == col)
return false;        // diagonal check        if(abs(board[i]
- col) == abs(i - row))            return false;
```

```
   }
   return true;
}
// Backtracking function void solveNQ(vector<int>& board,
int row, int N, int &count){    if(row == N){       count++;
       // print only first solution
if(count == 1){           cout <<
"First Solution:\n";           for(int i
= 0; i < N; i++)               cout <<
board[i] + 1 << " ";           cout <<
"\n\nBoard:\n";           for(int i =
0; i < N; i++){               for(int j = 0;
j < N; j++){                  if(board[i]
== j)                      cout << "Q ";

                 else
cout << ". ";
             }
             cout << endl;
          }
      }
     return;
   }
   for(int col = 0; col < N; col++){
if(isSafe(board, row, col, N)){
board[row] = col;           solveNQ(board,
row + 1, N, count);
      }
   }
```

```
}

int main(){    int N = 4;     vector<int> board(N, -1);

int count = 0;    cout << "N = " << N << endl;

solveNQ(board, 0, N, count);     cout << "\nTotal

Solutions = " << count << endl;     return 0;

}
```

Output:

```
N = 4
First Solution:
2 4 1 3

Board:
. Q . .
. . . Q
Q . . .
. . Q .

Total Solutions = 2
```

Practical 13: Sum Of Subsets

Aim: To find all subsets of a given set whose elements sum to a target value using backtracking.

Code:

```
#include <iostream> #include <vector> using

namespace std; void subsetSum(vector<int>& arr,

vector<int>& subset,

            int index, int target, int current){

   // If target achieved

if(current == target){

cout << "{ ";        for(int

x : subset)            cout

<< x << " ";        cout <<

"}" << endl;        return;
```

```
  }
  // Try remaining elements
for(int i = index; i < arr.size(); i++){
      // include element only if sum does not exceed target
     if(current + arr[i] <= target){
subset.push_back(arr[i]);          subsetSum(arr, subset, i +
1, target, current + arr[i]);          subset.pop_back(); //
backtrack
      }
  }
}
int main(){    vector<int> arr = {1, 2, 3, 4, 5};    int
target = 6;    cout << "Set    : { 1 2 3 4 5 }\n";    cout
<< "Target : " << target << "\n";    cout << "Subsets
with sum = " << target << ":\n";    vector<int>
subset;    subsetSum(arr, subset, 0, target, 0);
return 0;
}
```

Output:

```
Set    : { 1 2 3 4 5 }
Target : 6
Subsets with sum = 6:
{ 1 2 3 }
{ 1 5 }
{ 2 4 }
```

Practical 14: Naive String Match

Aim: To implement the Naive String Matching algorithm to find all occurrences of a pattern in a text and analyze its time complexity.

Code:

```
#include <iostream>
```

```
#include <string>
using namespace std; void
naiveSearch(string text, string pattern){
int n = text.size();     int m = pattern.size();
int count = 0;     cout << "Text    : " << text <<
endl;     cout << "Pattern : " << pattern <<
endl;     cout << "\nPattern found at
positions: ";     for(int i = 0; i <= n - m; i++){
      int j = 0;
      // check character by character
for(j = 0; j < m; j++){            if(text[i
+ j] != pattern[j])                break;
      }
      if(j == m){
cout << i << " ";
count++;
      }
   }
   if(count == 0)        cout << "None";     cout <<
"\nTotal occurrences: " << count << endl;
}
int main(){     string text =
"AABAACAADAABAABA";    string
pattern = "AABA";     naiveSearch(text,
pattern);

   return 0;
}
```

Output:

```
Text    : AABAACAADAABAABA
Pattern : AABA

Pattern found at positions: 0 9 12
Total occurrences: 3
```